Manufactured solutions for the eikonal–diffusion equation

Construction of the exact activation times, sources and reference functionals

Simão Castro

Table of contents

1 The problem **2**
1.1 Domain and governing equations . . . 2
1.2 Nomenclature . . . 2
1.3 Reading and regenerating . . . 4
1.4 Creating a new manufactured solution . . . 4

2 Cartesian eikonal field **7**
2.1 Description and limitations . . . 7
2.2 Problem instance . . . 7
2.3 Construction . . . 8
2.4 Exact solution . . . 8
2.5 Self-checks . . . 8
2.6 Figures . . . 9
2.7 Regeneration . . . 10

3 Eikonal-driven ECG **10**
3.1 Description and limitations . . . 10
3.2 Problem instance . . . 10
3.3 Construction . . . 11
3.4 Exact solution . . . 11
3.5 Self-checks . . . 12
3.6 Figures . . . 13
3.7 Regeneration . . . 13

4 Sealed spherical shell **14**
4.1 Description and limitations . . . 14
4.2 Problem instance . . . 14
4.3 Construction . . . 15
4.4 Exact solution . . . 16
4.5 Self-checks . . . 16
4.6 Figures . . . 17
4.7 Regeneration . . . 18

References **18**

1 The problem

This document describes how each manufactured solution of the eikonal–diffusion equation used by cardiacFoam is built. In the method of manufactured solutions (Roache 2002) an exact field is chosen first and the source that makes it satisfy the equation is derived from it. The document gives the exact activation times, the sources that make them exact, the reference values of the eikonal-driven ECG and the checks that each construction passes before anything is written. It states the problems only; it contains no solver output. Every displayed field, source and coefficient below is printed by the sympy generator that also writes the NumPy reference module and, for cardiacFoam, the C++ reference header, so the document, the references and the solver cannot disagree. A reader with this document and the generator folder can rebuild each problem in another code.

1.1 Domain and governing equations

The activation time $\psi(\mathbf{x})$ solves the anisotropic eikonal–diffusion equation (Colli Franzone, Guerri, and Rovida 1990; Jacquemet 2012) in the myocardium $\Omega \subset \mathbb{R}^d$,

$$-\nabla \cdot (\mathbf{M}\nabla\psi) + c_0\sqrt{\nabla\psi \cdot \mathbf{M}\nabla\psi} = 1 + S_\psi, \qquad \mathbf{M} = \frac{\boldsymbol{\sigma}}{\chi C_m},$$

where S_ψ is the manufactured source ($S_\psi = 0$ in ordinary runs) and c_0, in $\mathrm{s}^{-1/2}$, is a scaling coefficient rather than a conduction velocity. Ordinary runs seal the walls, $\mathbf{n} \cdot \mathbf{M}\nabla\psi = 0$, and prescribe $\psi = 0$ at the stimulus sites. Each manufactured problem states its own boundary and interior constraints: the Cartesian cube prescribes the exact ψ on the boundary, and the shell seals both walls and pins a small ball of cells about the stimulus point. The source is the residual of the operator on the exact field, $S_\psi = -\nabla \cdot (\mathbf{M}\nabla\psi) + c_0\sqrt{\nabla\psi \cdot \mathbf{M}\nabla\psi} - 1$.

The normalised cases use the solver's dimensional convention: ψ in s, $\mathbf{M}$ in $\mathrm{m}^2\ \mathrm{s}^{-1}$ and c_0 in $\mathrm{s}^{-1/2}$, with the fields written on the normalised coordinates $x/(1\,\mathrm{m})$, so the arguments of the exponential and square-root functions are dimensionless.

The eikonal-driven ECG observes ψ through a transmembrane template U (Pezzuto et al. 2017): $V_m(\mathbf{x}, t) = U(t - \psi(\mathbf{x}))$, so $\nabla V_m = -U'(t - \psi)\nabla\psi$, substituted in the pseudo-ECG functional (Gima and Rudy 2002) with the normalisation $c_{\mathrm{ECG}} = 1$.

1.2 Nomenclature

The code name is the key of the cardiacFoam case dictionaries or the name used in the generators and the NumPy reference modules. "Case units" marks a quantity defined only through the manufactured construction, with no physical calibration.

Symbol	Meaning	Unit	Code name
$\mathbf{x} = (x, y, z)$	position, normalised by 1 m	m	`x`, `y`, `z`
d, Ω, $\mathbf{n}$	spatial dimension; the domain; outward unit normal of $\partial\Omega$	–; –; –	`dim`, `d`
ψ	activation time	s	`activationTime`; `psi` (generators)
t, s	time; argument of the template	s	`t`, `s`

Symbol	Meaning	Unit	Code name
V_m	template transmembrane potential $U(t-\psi)$	case units	–
$\boldsymbol{\sigma}$	conductivity tensor	S m^{-1}	`conductivity`; `G` (shell generators), `sigma` (NumPy)
χ	membrane surface-to-volume ratio	m^{-1}	`chi`
C_m	membrane capacitance per unit area	F m^{-2}	`cm`
$\mathbf{M}$	eikonal metric, $\boldsymbol{\sigma}/(\chi C_m)$	m^2 s^{-1}	`M`
c_0	eikonal–diffusion scaling coefficient	s$^{-1/2}$	`c0`
S_ψ	manufactured source of the eikonal equation	–	`source`
$\mathbf{k}_d$, K_M	direction of the Cartesian field; $\mathbf{k}_d \cdot \mathbf{M}\mathbf{k}_d$	m^{-1}; s^{-1}	`k_vector(d)`; `K_M`
r_1, r_2	inner and outer radius of the shell	m	`r1`, `r2`
λ, ω	transmural coordinate; function vanishing on both spheres	–	`lam`, `om`
$\mathbf{f}, \mathbf{s}, \mathbf{n}_s$	fibre, sheet and sheet-normal directions	–	(shell generator)
$\mathbf{e}_a, \mathbf{e}_b$	fixed frame of the shell fibre rotation	–	(shell generator)
$\sigma_f, \sigma_s, \sigma_n$	conductivities along $\mathbf{f}$, $\mathbf{s}$, $\mathbf{n}_s$	S m^{-1}	`gil`, `git`, `gin`
θ_0, θ_1, γ_1, γ_2	fibre angles; rotation angles of the fixed frame	deg	`theta0`, `theta1`, `gamma1`, `gamma2`
$\mathbf{x}_s$, $\mathbf{M}_s$	stimulus point; $\mathbf{M}(\mathbf{x}_s)$	m; m^2 s^{-1}	`xs`; `Msi` holds $\mathbf{M}_s^{-1}$
δ	smoothing length of the anisotropic distance	case units	`delta`
r_{pin}	radius of the ball of pinned cells	m	`rPin`
$\hat{h}$, m_j, m	unsealed field; wall correction of sphere j; blend	–	(shell generator)
d_j, $\mathbf{n}_j$	$\partial_n\omega$ on sphere j; unit normal of sphere j out of the heart	–	(shell generator)
U, U'	transmembrane template and its derivative	case units	`template`
H, w	smooth step of the single-event template; its width	–; s	(eikonal ECG generator)
a, b	$\tanh(s/w)$, $\tanh((s-D)/w)$	–	(eikonal ECG generator)
D, T_r	duration and 10–90% rise time of the single-event template	s	`templateDuration`, `templateRise`; `duration`, `rise`

Symbol	Meaning	Unit	Code name
$\mathbf{r}_e$, Φ_e	electrode position; eikonal-driven ECG functional	m; case units	`electrodePositions`; `eikonalECG`
$\mathbf{r}$, φ_d	$\mathbf{x} - \mathbf{r}_e$; integrand of Φ_e in dimension d	m; case units	`integrand_d<d>_<template>`
c_{ECG}	normalisation of the ECG functional, here 1	–	–
q_{ref}, q	Gauss–Legendre points per axis of the reference quadrature (cube; shell)	–	`referenceQuadratureOrder`
W_q, κ	quadrature weights; kernel $\mathbf{r} \cdot \boldsymbol{\sigma} \nabla\psi / \lVert\mathbf{r}\rVert^3$ at the quadrature points	–; case units	–
μ, ϑ	$\cos\vartheta$; polar angle of the shell quadrature	–	–

1.3 Reading and regenerating

The generators live in `tutorials/manufacturedSolutions/generators/` of the cardiacFoam repository. Each case has one sympy generator `<case>_mms.py` that writes `reference/<case>.py`, a NumPy-only module with the exact fields, sources and reference quadratures, and `latex/<case>_*.tex`, the fragments printed in this document. Set up the environment once and run a generator from that folder:

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python cartesian_eikonal_mms.py --emit
.venv/bin/python cartesian_eikonal_mms.py --self-check
```

`--emit` runs the self-checks first and writes nothing if one fails; `--self-check` runs them alone. The generators that own a cardiacFoam header also take `--check PATH` and `--write PATH`, needed only by cardiacFoam developers. The document is built with

```
quarto render docs/eikonal/eikonal_mms.qmd --to pdf
```

and its figures with `python docs/eikonal/figures.py`.

1.4 Creating a new manufactured solution

A new manufactured solution is a new generator in `tutorials/manufacturedSolutions/generators/`. The steps below go from a copy of the template to a case that cardiacFoam runs and checks.

1.4.1 Copy the template

`template_mms.py` is the smallest complete generator: a passive monodomain field on the unit square with a diagonal conductivity and no ionic current,

$$V_m = \sqrt{t+1}\cos(\pi x)\cos(\pi y)$$

$$S = \left(\frac{\chi C_m}{2\sqrt{t+1}} + \pi^2(\sigma_x+\sigma_y)\sqrt{t+1}\right)\cos(\pi x)\cos(\pi y)$$

$$\chi = 3, \quad C_m = 2, \quad \sigma_x = 0.13, \quad \sigma_y = 0.0176$$

Copy it and its test, rename both, and edit every step marked `REPLACE`: `REFERENCE_NAME`, the coordinates, the constants, `exact_field`, `operator`, the self-checks and the emitted fragments.

```
cp template_mms.py mycase_mms.py
cp test_template_mms.py test_mycase_mms.py
```

1.4.2 Choose the exact field

Choose the field so that the boundary condition holds by construction; the solver's boundary treatment is then measured, not assumed. The generators of this folder use four patterns:

- a field whose gradient vanishes on every face, so every conormal flux is zero for any tensor (rotated cube, $\sin^2$ factors);
- a cosine eigenfunction of the operator with zero normal derivative on the faces of a box (FDA cube, the template);
- a sealing correction $F = \hat{h} + \omega m$ on a curved wall: ω vanishes on the walls and m cancels the conormal derivative of $\hat{h}$ there (the "Sealed spherical shell" section of this document);
- Dirichlet data plus pinned cells: the exact field is imposed on every face and on a layer of cells, so no flux condition is needed (Cartesian eikonal); this leaves the wall treatment untested.

Keep every factor smooth beyond the domain: polyhedral meshes put cell and face centres slightly outside a curved wall.

1.4.3 Derive the source

Write the operator the solver implements, with its signs and scalings, and take the source as its residual on the exact field, symbolically. For the monodomain equation that means the full tensor divergence $\nabla\cdot(\boldsymbol{\sigma}\nabla V_m)$, including the derivatives of a spatially varying tensor, the χC_m scaling of the time derivative, and the capacitance-normalised ionic current $I_{\text{ion}} = J_{\text{ion}}/C_m$ that enters as $-\chi C_m I_{\text{ion}}$. A source derived for a different convention converges to the wrong field.

1.4.4 Write the self-checks first

Each check fails before anything is written. The traps met in this folder, with the check that catches each:

trap	check
residual nonzero because a constant is rounded	keep constants as exact rationals; where a rounded constant is part of the problem, check the residual against a stated tolerance (FDA β_d)
conormal flux not zero on a curved wall	evaluate $\mathbf{n} \cdot \boldsymbol{\sigma} \nabla F$ on each wall at high precision (30 digits)
denominator that crosses zero	minimum of every denominator over the domain, extended past the walls, is positive
eikonal critical points	locate them; the Morse count (minima − saddles + maxima) on each sphere is 2 and every point is non-degenerate
electrode or quadrature node on a singular point	a coincident point contributes zero, with the solver's `VSMALL` guard reproduced in the generator
float sums that change with the Python version	`math.fsum` for every sum of floats
cancellation in a quadrature or finite-difference check	scale the tolerance by the sum of the absolute terms, not by the result
printed constant that hides digits	`printed_constant(value, digits)` asserts the printed decimal round-trips

Add a finite-difference evaluation of the operator on the exact field and compare it with the source; it catches an operator written for the wrong convention, which the symbolic residual cannot.

1.4.5 Emit and document

`python mycase_mms.py --emit` writes `reference/mycase.py` and `latex/mycase_*.tex`. Add a section to the document of the equation by copying an existing case section and keeping its order: Description and limitations, Problem instance, Construction, Exact solution, Self-checks, Figures, Regeneration. Print every formula by including its fragment from `../../latex/`, as the existing sections do; the release test fails if an emitted fragment is included nowhere. Add the figures as a function of `docs/<equation>/figures.py`, add the generator to `GENERATORS` of `test_release.py` and to `CLI_GENERATORS` of `test_mms_codegen.py`, and render:

```
python docs/monodomain/figures.py
quarto render docs/monodomain/monodomain_mms.qmd --to pdf
```

1.4.6 Use it in cardiacFoam

1. Write or extend a verifier in `src/verificationModels/<equation>Verification/` that evaluates the reference on the mesh and reports the error norms.
2. Generate its reference header with `--write PATH`: either a whole header, or the lines between `// BEGIN GENERATED <name>` and `// END GENERATED <name>` of a hand-written one. Map the header name to its region function in `HEADERS` of the generator (`HEADERS = {}` in the template).

3. Add the case to `parity/mmsParity.C` and `CASES` of `parity/test_parity.py`, and a `test_parity_<case>` to the generator test that compares the compiled header with `reference/mycase.py`.
4. Add a tutorial case in `tutorials/manufacturedSolutions/<equation>/<geometry>/` with `Allrun`, a `regression/regressionTest.sh` on `tutorials/regression/lib.sh` and its reference. List it in the README of its equation folder and in the table of `tutorials/README.md`, and add its row to "Cases that use each generator" in `generators/README.md`.
5. Check the header, the parity and the regression:

```
python mycase_mms.py --check PATH
python -m pytest parity/test_parity.py test_mycase_mms.py
```

   then run `regression/regressionTest.sh` of the tutorial case.

1.4.7 Reproduce

A new generator is reproduced like the existing ones: set up the environment, run `--self-check` and `--emit`, the tests, the slow release test and the renders, as in the previous section. The release test then runs the new generator from a fresh environment and checks that its emitted files are unchanged.

2 Cartesian eikonal field

2.1 Description and limitations

An exponential activation time on the unit cube $[0,1]^d$, $d = 1, 2, 3$, with a constant conductivity tensor, full or diagonal. The case exercises the interior discretisation of the eikonal–diffusion operator: the metric term, the anisotropic diffusion term with a full tensor, the cell-gradient reconstruction and the outer nonlinear iteration.

The exact ψ is prescribed on every face and on the cells whose centre has the smallest x, y or z, so the case exercises neither a sealed wall, nor the zero-gradient outer boundary of ordinary runs, nor the placement of internal stimulus sites. The tensor is constant, so its derivatives do not enter the source.

2.2 Problem instance

Unit cube $[0,1]^d$ with

$$\chi = 3, \quad C_m = 2, \quad c_0 = 1$$

and one of the two tensors of the tutorial,

$$\boldsymbol{\sigma}_{\text{rotated}} = \begin{bmatrix} 0.106875 & -0.0084931 & -0.022561 \\ -0.0084931 & 0.116682 & -0.0130256 \\ -0.022561 & -0.0130256 & 0.0398782 \end{bmatrix}$$

$$\boldsymbol{\sigma}_{\text{axis}} = \begin{bmatrix} 0.111453302 & 0 & 0 \\ 0 & 0.12158542 & 0 \\ 0 & 0 & 0.030396355 \end{bmatrix}$$

In cardiacFoam this is the tutorial `eikonal/cube` (`-conductivity rotated`, the default, or `axis`; `-dim 1|2|3`) with the verifier `manufacturedEikonalVerifier`.

2.3 Construction

The field is the exponential of a linear form,

$$\mathbf{k}_1 = (1, 0, 0)\,, \quad \mathbf{k}_2 = (1, 1, 0)\,, \quad \mathbf{k}_3 = (1, 1, 1)$$

chosen because it reproduces itself under differentiation: $\nabla\psi = \mathbf{k}_d\psi$. With $\mathbf{M}$ constant, $\nabla\psi \cdot \mathbf{M}\nabla\psi = K_M\psi^2$ and $\nabla \cdot (\mathbf{M}\nabla\psi) = K_M\psi$, so with $\psi > 0$ the left-hand side of the equation equals $\psi(c_0\sqrt{K_M} - K_M)$ and the source follows in closed form. The construction requires $K_M > 0$; a tensor that is not positive along $\mathbf{k}_d$ is rejected (the C++ verifier rejects $K_M \leq 10^{-300}$).

2.4 Exact solution

$$\psi = e^{x}$$

$$\psi = e^{x+y}$$

$$\psi = e^{x+y+z}$$

$$\nabla\psi = \mathbf{k}_d\psi$$

$$K_M = \mathbf{k}_d \cdot \mathbf{M}\,\mathbf{k}_d$$

$$S_\psi = \psi\left(\sqrt{K_M}c_0 - K_M\right) - 1$$

The full expressions are `fields(d, sigma, chi, cm, c0)` of `cartesian_eikonal_mms.py`, and `tau_d<d>`, `grad_tau_d<d>`, `source_d<d>` and the checked wrapper `source(d, point, sigma, c0, chi, cm)` of `reference/cartesian_eikonal.py`.

2.5 Self-checks

`--self-check` asserts that the residual of the eikonal equation with the source vanishes to 10^{-12} at a sample point for $d = 1, 2, 3$, for both tutorial tensors and for $c_0 \in \{1, 5, 66\}$; and that a tensor with $K_M = 0$ is rejected.

2.6 Figures

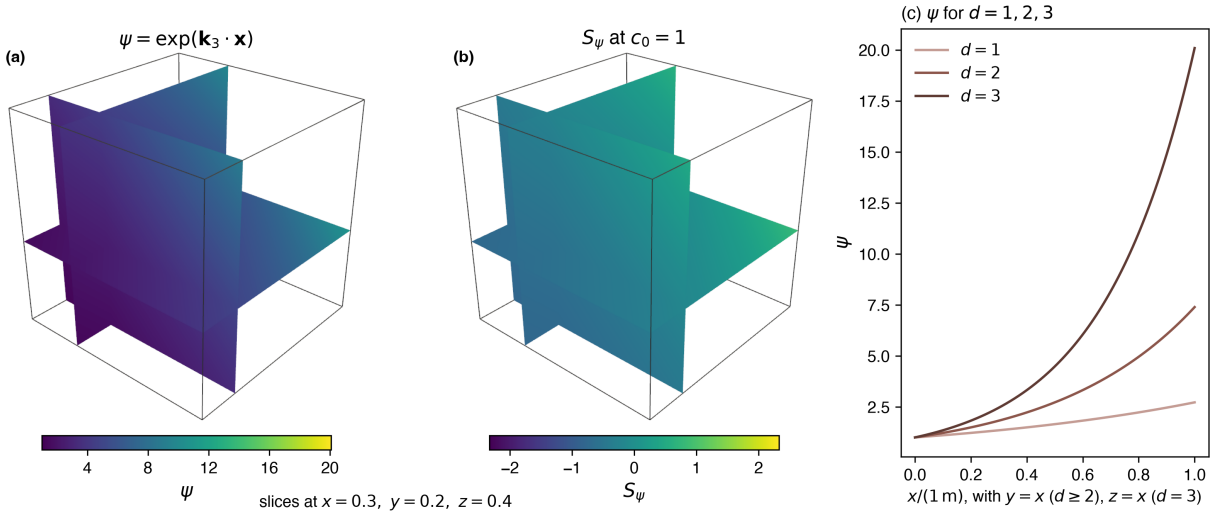
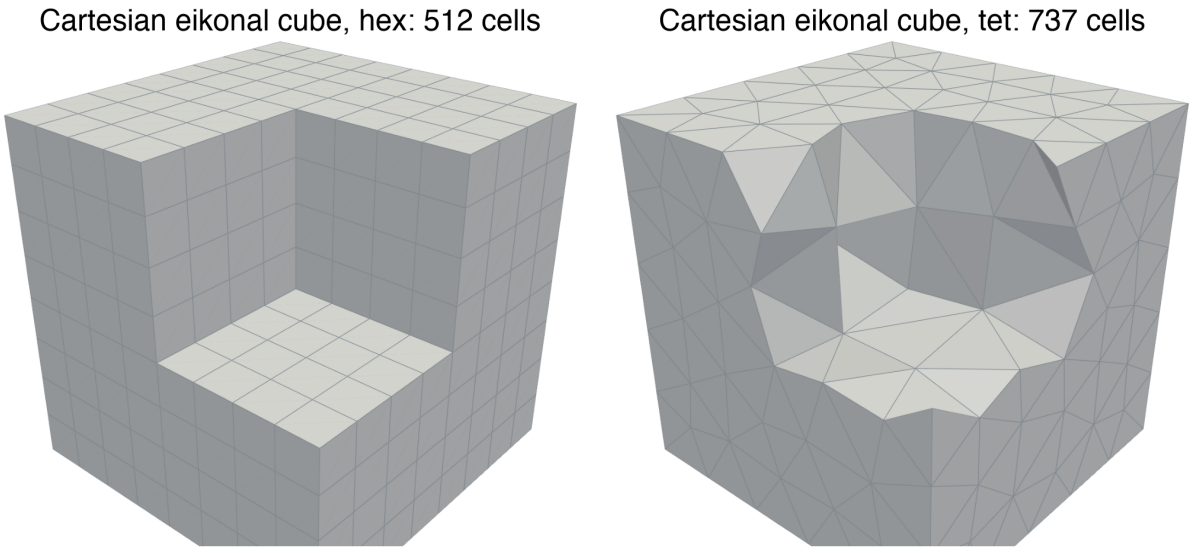


Figure 1: Cartesian eikonal field. (a) $\psi = \exp(\mathbf{k}_3 \cdot \mathbf{x})$ on three orthogonal slices of the unit cube. (b) The source S_ψ at $c_0 = 1$ with the rotated tensor on the same slices. (c) ψ of $d = 1, 2, 3$ along the diagonal of the domain.



(a) hex: `-mesh hex -N 8`

(b) tet: `-mesh tet -N 5`

Figure 2: Meshes of the unit cube from the tutorial `eikonal/cube` (`-dim 3`), made by the meshing steps of its `Allrun` at a coarse resolution chosen for legibility (the tutorial default is $N = 20$): (a) `blockMesh` with $N = 8$ cells per unit length; (b) `gmsh` tetrahedra of element size 1/5, seed 1, converted by `gmshToFoam`. The octant facing the viewer is removed cell by cell.

2.7 Regeneration

```
python cartesian_eikonal_mms.py --emit
python cartesian_eikonal_mms.py --self-check
H=../../../src/verificationModels/eikonalVerification
python cartesian_eikonal_mms.py --check $H/manufacturedEikonalReference.H
```

`--emit` writes `reference/cartesian_eikonal.py` and `latex/eikonal_*.tex`. The header regions are $\mathbf{k}_d$, ψ and $\nabla\psi$, K_M and S_ψ; $\mathbf{M} = \boldsymbol{\sigma}/(\chi C_m)$ and the positivity guard on K_M are hand-written.

3 Eikonal-driven ECG

3.1 Description and limitations

The eikonal-driven ECG of the Cartesian field of Section 2: the pseudo-ECG functional of the template voltage $V_m = U(t - \psi)$ at electrodes outside the cube. The case exercises the chain from the computed activation time through its reconstructed gradient and the template to the functional; its reference uses the exact gradient $\nabla\psi = \mathbf{k}_d\psi$. The reference is a quadrature of the exact integrand, not a closed form. The generator covers the cube; the shell ECG reference of cardiacFoam (`domain shell`) is the same integrand over the shell of Section 4, computed by the verifier from the shell header and not part of the emitted modules. The sine template repeats many times across the activation range, so its integrand changes sign many times over the cube and the integral cancels to a small fraction of the sum of the magnitudes of its terms.

3.2 Problem instance

Two templates: a single smooth event and a sine. The single event is

$$U(s) = H(s)\,(1 - H(s - D))$$

$$H(s) = \frac{\tanh\left(\frac{s}{w}\right)}{2} + \frac{1}{2}$$

$$w = \frac{T_r}{2\,\mathrm{atanh}\left(\frac{4}{5}\right)}$$

so that H rises from 10% to 90% within the rise time T_r, with

$$D = 64, \quad T_r = 4, \quad q_{\mathrm{ref}} = 48$$

The duration D exceeds the activation range $e^3 - 1$ of $d = 3$, so $U'(t - \psi)$ keeps one sign across the cube. The sine template is

$$U(s) = \sin(2\pi s)$$

In cardiacFoam:

- the ECG of the tutorial `eikonal/cube`;
- verifier `manufacturedEikonalECGVerifier`;
- `template singleEvent` (the default) or `template sine`.

3.3 Construction

With $\mathbf{r} = \mathbf{x} - \mathbf{r}_e$, and in $d < 3$ only the first d coordinates of $\mathbf{r}$, the functional is the integral over $[0,1]^d$ of

$$\varphi_d = -\frac{U'\mathbf{r} \cdot \boldsymbol{\sigma} \nabla \psi}{\|\mathbf{r}\|^3}$$

with U' evaluated at $t - \psi(\mathbf{x})$. With $a = \tanh(s/w)$ and $b = \tanh((s-D)/w)$, the template derivatives are

$$a = \tanh\left(\frac{s}{w}\right), \quad b = \tanh\left(\frac{-D+s}{w}\right)$$

$$U'(s) = \frac{(1-a^2)(1-b) - (1-b^2)(a+1)}{4w}$$

$$U'(s) = 2\pi \cos(2\pi s)$$

3.4 Exact solution

The reference $\Phi_e(\mathbf{r}_e, t)$ is a tensor-product Gauss–Legendre quadrature with q_{ref} points per axis on $[0,1]^d$. The functional has no separate time factor: the time enters through $U'(t-\psi)$ at every quadrature point. The NumPy module holds `integrand_d<d>_<template>(x, y, z, t, ex, ey, ez, sxx, sxy, sxz, syy, syz, szz, duration, rise)`, `template_derivative_single_event`, `template_derivative_sine` and `spatial_reference(d, template, sigma, electrode, t, order, duration, rise)`.

Note for implementers. The tutorial uses $q_{\text{ref}} = 48$. The integrand is set to 0 where $\|\mathbf{r}\|^2 \le 10^{-300}$ ($|x - r_{e,x}| \le 10^{-300}$ in $d = 1$), so an electrode that coincides with a quadrature point contributes 0, not a non-finite value. Because the quadrature sum cancels (the functional can be 10^{-6} of the sum of the magnitudes of its terms) and tanh and cos are evaluated in a different order by different codes, two correct implementations agree only to a bound that scales with those magnitudes, not to a fixed relative tolerance. cardiacFoam compares its C++ reference with the NumPy reference within $10^{-12}|\Phi_e| + 2 \times 10^{-13} \sum_q |W_q\, U'\, \kappa| + 3 \times 10^{-15} \sum_q |W_q\, \kappa|$, with W_q the quadrature weights and $\kappa = \mathbf{r} \cdot \boldsymbol{\sigma} \nabla \psi / \|\mathbf{r}\|^3$ the kernel at the quadrature points. For the shell, cardiacFoam integrates the same kernel with q Gauss–Legendre nodes in the radius $|\mathbf{x}|$ (Jacobian $|\mathbf{x}|^2$) and in $\mu = \cos\vartheta$, with ϑ the polar angle, and $2q$ trapezoid nodes in the azimuth.

3.5 Self-checks

`--self-check` asserts that the integrand equals the Cartesian component form of the C++ reference for $d = 1, 2, 3$ and both templates, to 10^{-12}; that it vanishes at an electrode on the evaluation point; that the reference at a sample electrode agrees between orders 48 and 96 to $10^{-10} \max(1, |\Phi_e|)$, an absolute bound for $|\Phi_e| \leq 1$ and a relative one above; and that the printed single-event building blocks (H, U, w, a, b) give the same U' as the template derivative of the generator and as the derivative of U.

3.6 Figures

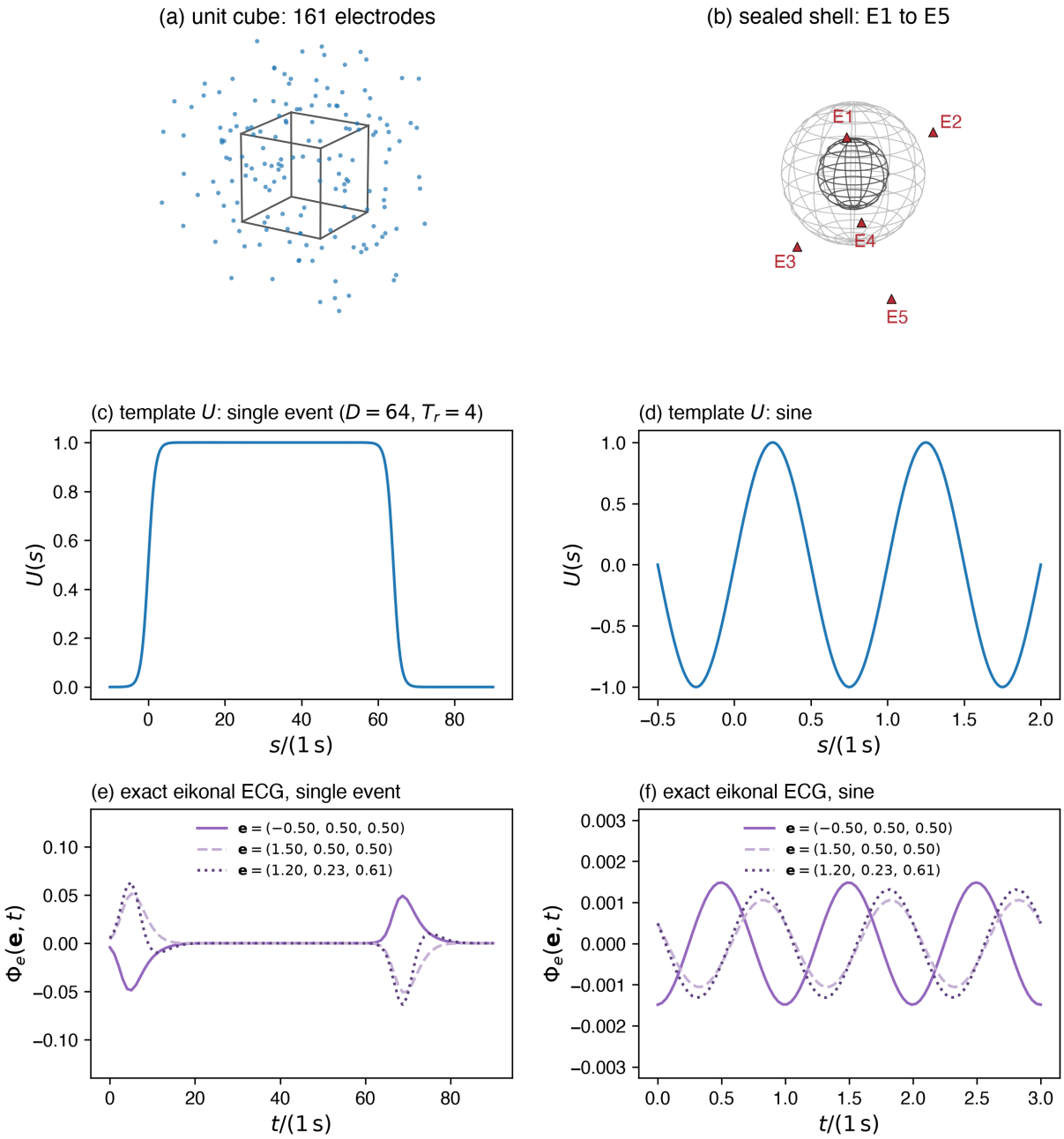


Figure 3: Eikonal-driven ECG. (a) The 161 electrodes of the cube tutorial. (b) The five named electrodes of the shell tutorial. (c) The single-event template U. (d) The sine template. (e, f) The reference $\Phi_e(\mathbf{r}_e, t)$ of the Cartesian field ($d = 3$) at three electrodes for each template. The figure writes $\mathbf{e}$ for the electrode position $\mathbf{r}_e$.

3.7 Regeneration

```
python eikonal_ecg_mms.py --emit
python eikonal_ecg_mms.py --self-check
H=../../../src/verificationModels/eikonalVerification
python eikonal_ecg_mms.py --check $H/manufacturedEikonalReference.H
```

`--emit` writes `reference/eikonal_ecg.py` and `latex/eikonal_ecg_*.tex`. The header region is the kernel of the C++ integrand for $d = 1, 2, 3$; the guards, $-\boldsymbol{\sigma}\nabla\psi\, U'$ and the template derivatives are hand-written.

4 Sealed spherical shell

4.1 Description and limitations

The heart-like shell $r_1 < |\mathbf{x}| < r_2$ with both spheres sealed, the rotating anisotropic conductivity of the monodomain shell, and an activation time spreading from a mid-wall stimulus point. The exact ψ has non-zero normal and tangential gradients on both walls while $\mathbf{n} \cdot \mathbf{M}\nabla\psi = 0$ holds there exactly, and its wavefronts collide on the walls behind the cavity. The case poses the full sealed-wall problem: curved walls, a rotating anisotropic tensor oblique to both walls, and a spatially varying metric whose derivatives enter the source. At the critical points of ψ on the walls the metric $\sqrt{\nabla\psi \cdot \mathbf{M}\nabla\psi}$ is not differentiable, so the source is finite there but not smooth; this is a feature of the solution.

The sealed problem fixes ψ only up to a constant, so the cells near the stimulus point are pinned to the exact ψ: the case does not exercise a stimulus region or the constraint $\psi = 0$ of ordinary runs.

The construction is parametric in c_0. The stated problem instance is $c_0 = 1$; a higher c_0 lies outside the regime in which the solver paper verifies the shell, and the document makes no statement about it.

4.2 Problem instance

$$r_1 = 0.5, \quad r_2 = 1, \quad \gamma_1 = 20^\circ, \quad \gamma_2 = 50^\circ, \quad \sigma_f = 0.17,$$
$$\sigma_s = 0.019, \quad \sigma_n = 0.01, \quad \theta_0 = -20^\circ, \quad \theta_1 = 90^\circ, \quad \chi = 3,$$
$$C_m = 2, \quad c_0 = 1, \quad \delta = 0.05, \quad r_{\text{pin}} = 0.15$$

with the stimulus point

$$\mathbf{x}_s = 0.75\, \frac{(1, 0.3, 0.2)}{\|(1, 0.3, 0.2)\|} \approx (0.70554, 0.21166, 0.14111)$$

The verifier pins the cells whose centre lies within r_{pin} of $\mathbf{x}_s$ to the exact ψ (the nearest cell when the mesh has no centre in that ball) and imposes no value on a wall. In cardiacFoam this is `eikonal/sphericalShell` with the verifier `manufacturedShellEikonalVerifier`; its `Allrun` takes `-c0 <value>` (default 1). The verifier reads c_0 from the case, so the source follows the value given.

4.3 Construction

Geometry and conductivity. λ, ω, the fibre frame, the tensor $\boldsymbol{\sigma}$, the wall normals $\mathbf{n}_j$ and d_j are those of the monodomain shell:

$$\lambda = \frac{-r_1^2 + |\mathbf{x}|^2}{-r_1^2 + r_2^2}$$

$$\theta = \lambda\theta_1 + \theta_0$$

$$[\mathbf{e}_a\ \mathbf{e}_b\ \mathbf{n}_s] = \begin{bmatrix} \cos(\gamma_1) & -\sin(\gamma_1) & 0 \\ \sin(\gamma_1) & \cos(\gamma_1) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\gamma_2) & -\sin(\gamma_2) \\ 0 & \sin(\gamma_2) & \cos(\gamma_2) \end{bmatrix}$$

$$\mathbf{f} = \mathbf{e}_a \cos(\theta) + \mathbf{e}_b \sin(\theta)$$

$$\mathbf{s} = -\mathbf{e}_a \sin(\theta) + \mathbf{e}_b \cos(\theta)$$

$$\boldsymbol{\sigma} = \mathbf{f} \otimes \mathbf{f}\sigma_f + \sigma_n \mathbf{n}_s \otimes \mathbf{n}_s + \sigma_s \mathbf{s} \otimes \mathbf{s}$$

$$\omega = \frac{\left(-r_1^2 + |\mathbf{x}|^2\right)\left(-r_2^2 + |\mathbf{x}|^2\right)}{(r_1 + r_2)\left(-r_1^2 + r_2^2\right)}$$

$$\mathbf{n}_1 = -\frac{\mathbf{x}}{r_1}$$

$$\mathbf{n}_2 = \frac{\mathbf{x}}{r_2}$$

$$d_1 = \frac{2r_1}{r_1 + r_2}$$

$$d_2 = \frac{2r_2}{r_1 + r_2}$$

Unsealed field. The smoothed anisotropic distance from $\mathbf{x}_s$, measured in the metric frozen at the stimulus point, divided by c_0, with the smoothing length δ keeping it differentiable at $\mathbf{x}_s$:

$$\mathbf{M} = \frac{\boldsymbol{\sigma}}{\chi C_m}$$

$$\mathbf{M}_s = \mathbf{M}(\mathbf{x}_s)$$

$$\hat{h} = \frac{\sqrt{\delta^2 + (\mathbf{x} - \mathbf{x}_s) \cdot \mathbf{M}_s^{-1}(\mathbf{x} - \mathbf{x}_s)}}{c_0}$$

Sealing. The correction of the monodomain shell, written for the operator tensor $\mathbf{M}$:

$$m_j = -\frac{\left(\mathbf{n}_j \cdot \mathbf{M}\nabla\hat{h}\right)}{(\mathbf{n}_j \cdot \mathbf{M}\,\mathbf{n}_j)\,d_j}$$

$$m = \lambda m_2 + m_1\,(1-\lambda)$$

$$\psi = \hat{h} + m\omega$$

so that $\mathbf{n}_j \cdot \mathbf{M}\nabla\psi = 0$ on both spheres.

Dependence on c_0. $\hat{h}$ and m both carry the factor $1/c_0$, so ψ scales as $1/c_0$: its critical points and their types do not depend on c_0, while the source does. The checks can be run at any c_0 (`--self-check --c0 VALUE`).

4.4 Exact solution

The source is the residual of the operator on the exact field, including the derivatives of the spatially varying $\mathbf{M}$:

$$S_\psi = c_0\sqrt{\nabla\psi \cdot \mathbf{M}\nabla\psi} - \nabla\cdot(\mathbf{M}\nabla\psi) - 1$$

The expanded ψ, $\nabla\psi$ and source are not printed: they are `expressions()` of `shell_eikonal_mms.py` and `psi`, `gradPsi`, `source` and `sigma` of `reference/shell_eikonal.py`, functions of (x, y, z) and the parameters, with `field(name, x, y, z, **overrides)` to evaluate them with the default parameters; `c0` is one of those parameters.

4.5 Self-checks

`--self-check` (at $c_0 = 1$, or at `--c0 VALUE`) asserts:

- the stored $\mathbf{M}_s^{-1}$ inverts $\mathbf{M}(\mathbf{x}_s)$ to 10^{-12}, and $|\mathbf{x}_s| = 0.75$;
- sealing: $\max|\mathbf{n}\cdot\mathbf{M}\nabla\psi| < 10^{-25}$ at 200 points of each sphere in 30-digit arithmetic;
- the gradient against central differences of ψ to 10^{-5}, and the source against central differences of the flux to 10^{-6};
- $\mathbf{n}_j \cdot \boldsymbol{\sigma}\mathbf{n}_j > 0$ over the shell widened by 0.05, and the source finite at 2×10^5 points of the shell;
- critical points: on each sphere the Morse count (minima − saddles + maxima) of ψ is 2, so the wall critical points found are complete; every interior critical point lies in the pinned ball; every critical point outside the ball is non-degenerate (smallest to largest absolute Hessian eigenvalue above 10^{-3}); the local minima outside the ball have $\psi > 0$ and the pinned minimum exceeds -10^{-2}; the source is finite next to every critical point.

`--report [--c0 VALUE]` prints the table of critical points.

4.6 Figures

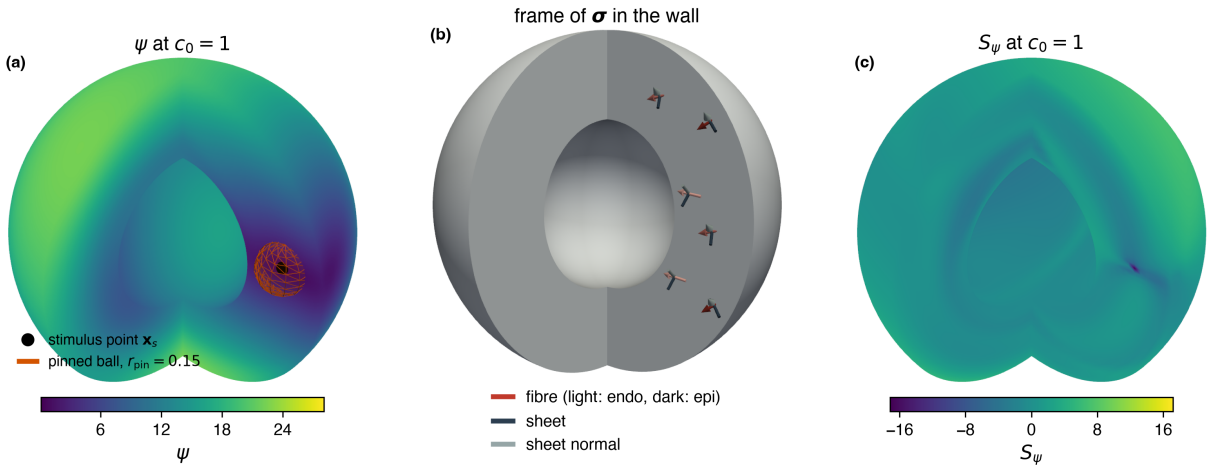


Figure 4: Sealed shell. (a) ψ on a cut-away of the shell with the stimulus point and the pinned ball of radius r_{pin}. (b) The fibre, sheet and sheet-normal directions through the wall. (c) The source S_ψ at $c_0 = 1$.

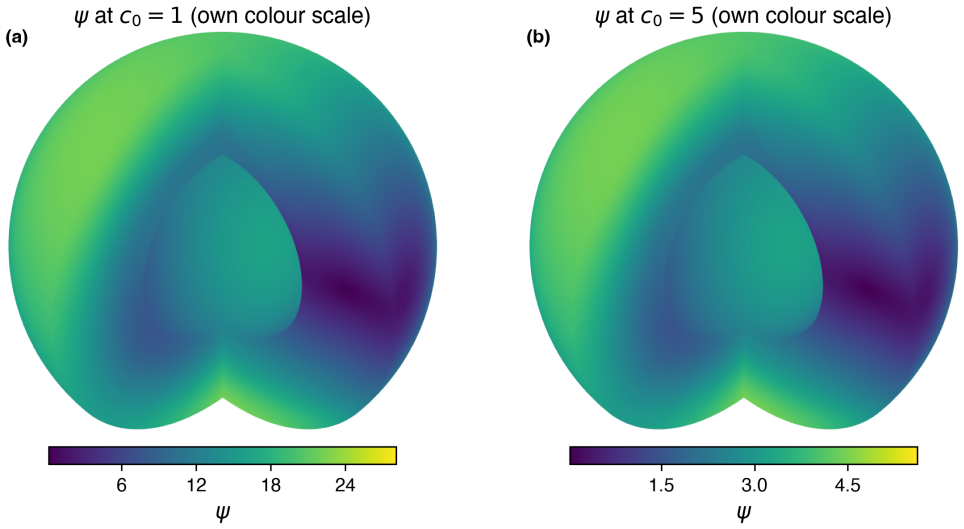
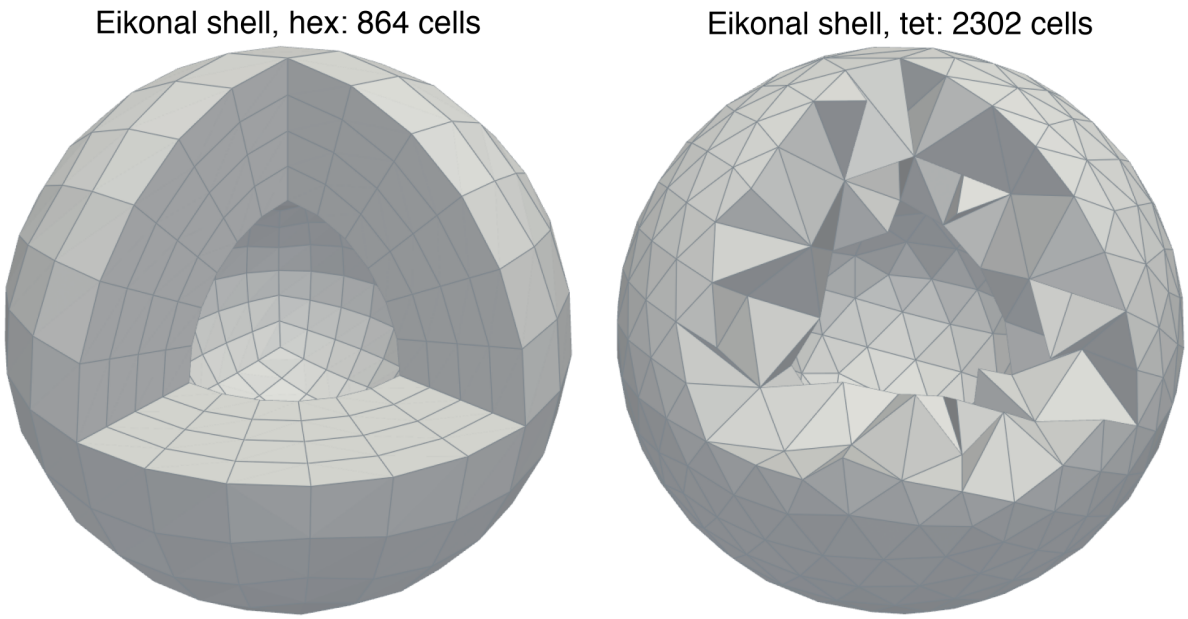


Figure 5: ψ at $c_0 = 1$ and $c_0 = 5$, each on its own colour scale: the two fields differ by the factor $1/c_0$.



(a) hex: `-mesh hex -N 8` **(b)** tet: `-mesh tet -N 5`

Figure 6: Meshes of the shell from the tutorial `eikonal/sphericalShell`, made by the meshing steps of its `Allrun` at a coarse resolution chosen for legibility (the tutorial default is $N = 12$): (a) the `blockMesh` cubed sphere with $N = 8$; (b) `gmsh` tetrahedra of element size 1/5, seed 1, converted by `gmshToFoam`. The octant facing the viewer is removed cell by cell, exposing the cavity.

4.7 Regeneration

```
python shell_eikonal_mms.py --emit
python shell_eikonal_mms.py --self-check [--c0 VALUE]
python shell_eikonal_mms.py --report [--c0 VALUE]
H=../../../src/verificationModels/eikonalVerification
python shell_eikonal_mms.py --check $H/manufacturedShellEikonalReference.H
```

`--emit` writes `reference/shell_eikonal.py` and `latex/shell_eikonal_*.tex`; the geometry fragments `latex/shell_*.tex` are written by `shell_monodomain_mms.py`. The header and the emitted module carry c_0 as a runtime parameter and are written at $c_0 = 1$.

References

Colli Franzone, Piero, Luciano Guerri, and Sergio Rovida. 1990. "Wavefront Propagation in an Activation Model of the Anisotropic Cardiac Tissue: Asymptotic Analysis and Numerical Simulations." *Journal of Mathematical Biology* 28 (2): 121–76. https://doi.org/10.1007/BF00163143.

Gima, Kazutaka, and Yoram Rudy. 2002. "Ionic Current Basis of Electrocardiographic Waveforms: A Model Study." *Circulation Research* 90 (8): 889–96. https://doi.org/10.1161/01.RES.0000016960.61087.86.

Jacquemet, Vincent. 2012. "An Eikonal-Diffusion Solver and Its Application to the Interpolation and the Simulation of Reentrant Cardiac Activations." *Computer Methods and Programs in Biomedicine* 108 (2): 548–58. https://doi.org/10.1016/j.cmpb.2011.05.003.

Pezzuto, Simone, Peter Kal'avský, Mark Potse, Frits W. Prinzen, Angelo Auricchio, and Rolf Krause. 2017. "Evaluation of a Rapid Anisotropic Model for ECG Simulation." *Frontiers in Physiology* 8: 265. https://doi.org/10.3389/fphys.2017.00265.

Roache, Patrick J. 2002. "Code Verification by the Method of Manufactured Solutions." *Journal of Fluids Engineering* 124 (1): 4–10. https://doi.org/10.1115/1.1436090.